

VHDL per circuiti sequenziali

Specificare memorie

- ▶ **Per ora abbiamo visto tutti oggetti combinatori**
 - ▶ Caratterizzati dalla mancanza di memoria
 - ▶ Quindi, per ogni valore degli ingressi, il codice deve specificare il valore delle uscite
- ▶ **I circuiti sequenziali devono essere in grado di memorizzare dei valori**
 - ▶ Le uscite devono poter dipendere dai loro valori precedenti
 - ▶ Basta usare un tipo **buffer**

```
-- Latch di tipo D
entity latch is
  port (d, c : in std_logic; q : buffer std_logic);
end entity latch;

architecture equazioni of latch is
begin
  q <= d when c = '1' else q;
end architecture equazioni;
```

Occorre ricordare il valore precedente di q, quindi serve una memoria

Edge triggered

- ▶ **Come far aggiornare un segnale solo al fronte di un clock?**
 - ▶ Occorre verificare che ci sia stato effettivamente un evento sul clock
 - ▶ Lo si può fare guardando gli *attributi* di un segnale
 - ▶ 'event: il segnale ha un evento nell'istante di simulazione
 - ▶ 'stable: il segnale non ha un evento nell'istante di simulazione
- ▶ **Quindi basta aggiornare q solo quando c è ad 1 ed ha un evento, cioè siamo al fronte di salita**
 - ▶ Così d può cambiare mentre $c = 1$, ma il valore di q non viene aggiornato

```
-- Flip flop di tipo D edge triggered
entity flip_flop is
  port (d, c : in std_logic; q : buffer std_logic);
end entity flip_flop;

architecture equazioni of flip_flop is
begin
  q <= d when c = '1' and c'event else q;  -- oppure and not c'stable
end architecture equazioni;
```

Funzioni pre-definite

- ▶ **L'edge triggered è così comune che si è pensato di definire una funzione apposta**
 - ▶ Si usa l'espressione `rising_edge( c )`
 - ▶ Ovviamente c'è anche `falling_edge( c )`
 - ▶ Nel `std_logic`, evitano di triggerare se c'è un evento da H a 1!
- ▶ **Facile aggiungere anche un reset**
 - ▶ Lo facciamo per esempio asincrono attivo basso
 - ▶ E se lo si volesse fare sincrono (che abbia effetto solo al fronte del clock)?

```
-- Flip flop di tipo D edge triggered
entity flip_flop is
  port (d, c, r : in std_logic; q : buffer std_logic);
end entity flip_flop;

architecture equazioni of flip_flop is
begin
  q <= '0' when r = '0' else
    d when rising_edge( c ) else
      q;
end architecture equazioni;
```

Processi sequenziali

Registri, contatori, etc.

Perché i modelli visti sono combinatori?

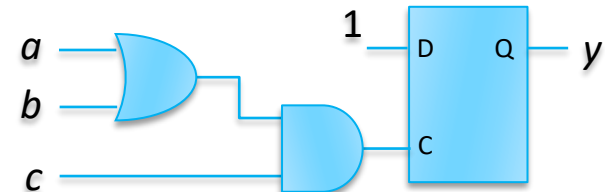
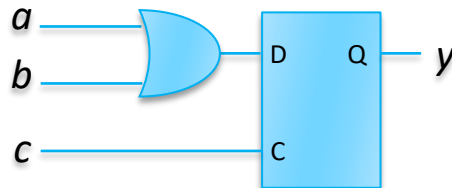
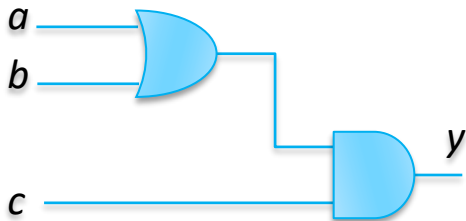
- ▶ Il circuito si comporta in maniera combinatoria se è sempre possibile assegnare un valore ai segnali senza dover ricordare il valore precedente
 - ▶ Quando si definisce un circuito **combinatorio**, occorre allora fare attenzione a fare **sempre un'assegnazione alle uscite per tutti i possibili valori degli ingressi**
 - ▶ **Se non lo si fa, gli strumenti di sintesi produrranno un latch o un flip flop per ricordare il valore precedente**, visto che uno nuovo non viene assegnato
 - ▶ Questo vale sia per lo stile data flow che per lo stile comportamentale
 - ▶ Le condizioni per le quali occorre ricordare vanno a formare il clock degli elementi sequenziali introdotti

Inferenza di latch

```
process (a, b, c) is
begin
  if (c = '1') then
    if (a = '1') or (b = '1') then
      y <= '1';
    else
      y <= '0';
    end if;
  else
    y <= '0';
  end if;
end process;
```

```
process (a, b, c) is
begin
  if (c = '1') then
    if (a = '1') or (b = '1') then
      y <= '1';
    else
      y <= '0';
    end if;
  else
  y <= '0';
  end if;
end process;
```

```
process (a, b, c) is
begin
  if (c = '1') then
    if (a = '1') or (b = '1') then
      y <= '1';
    else
    y <= '0';
    end if;
  else
  y <= '0';
  end if;
end process;
```



Per evitare errori

- ▶ Se si vuole fare un circuito combinatorio è sufficiente dare alle uscite dei valori di default all'inizio del processo
- ▶ In questo modo sono sicuramente sempre assegnate almeno una volta e non si inferisce un latch
- ▶ Se il default non è corretto basta fare una seconda assegnazione (tanto vale l'ultima!)

```
process (a, b, c) is
begin
  if (c = '1') then
    if (a = '1') or (b = '1') then
      y <= '1';
    else
      y <= '0';
    end if;
  else
    y <= '0';
  end if;
end process;
```

```
process (a, b, c) is
begin
  y <= '0';
  if (c = '1') then
    if (a = '1') or (b = '1') then
      y <= '1';
    end if;
  end if;
end process;
```


Registri

► Latch di tipo D

- Il latch è sensibile sia all'ingresso di dato d , sia al clock clk
 - Il processo è sensibile sia alle variazioni di d sia alle variazioni di clk
 - Si assegna a q il valore di d solo quando clk vale 1
 - Se d cambia e clk è 0, allora il circuito deve fornire il valore precedente di q
 - Si crea quindi una memoria
- ## ► Notare di nuovo la sensitivity list
- Contiene sia d sia clk

```
-- Latch di tipo D
library ieee;
use ieee.std_logic_1164.all;

entity D_latch is
    port (d, clk : in std_logic;
          q : out std_logic);
end entity D_latch;

architecture behavioral of D_latch is
begin
    process (d, clk) is
    begin
        if clk = '1' then
            q <= d;
        end if;
    end process;
end architecture behavioral;
```

Registri edge triggered

► Flip flop di tipo D

- Il codice è simile a quello precedente
- Il processo però è sensibile solo al segnale *clk*, e non al segnale *d*
- Che non vuol dire che *q* non dipenda da *d*, ma solo che il valore di *q* cambia solo in corrispondenza delle variazioni di *clk*
- In questo modo il valore di *q* viene aggiornato solo quando *clk* esegue una transizione
- Poiché si verifica che *clk* sia 1 prima di fare l'assegnazione, questa avviene solo al fronte di salita

```
-- Flip flop di tipo D
library ieee;
use ieee.std_logic_1164.all;

entity D_FF is
    port (d, clk : in std_logic;
          q : out std_logic);
end entity D_FF;

architecture behavioral of D_FF is
begin
    process (clk) is
    begin
        if clk = '1' then
            q <= d;
        end if;
    end process;
end architecture behavioral;
```

Registri edge triggered

- ▶ **Flip flop di tipo D con reset asincrono**
 - ▶ Aggiungiamo un *reset* attivo basso alla interfaccia
 - ▶ Il processo è sensibile sia al clock che al reset
 - ▶ Per il segnale *clk* dobbiamo però ora controllare esplicitamente che ci sia un fronte
 - ▶ Se usassimo il codice di prima, si potrebbe assegnare *d* a *q* se *clk* è 1 e *res* va da 0 a 1

```
-- Flip flop di tipo D con reset async
library ieee;
use ieee.std_logic_1164.all;

entity D_FF is
    port (d, clk, res : in std_logic;
          q : out std_logic);
end entity D_FF;

architecture behavioral of D_FF is
begin
    process (clk, res) is
    begin
        if res = '0' then
            q <= '0';
        elsif rising_edge(clk) then
            q <= d;
        end if;
    end process;
end architecture behavioral;
```

Registri edge triggered

- ▶ Flip flop di tipo D con Set e Reset asincroni
 - ▶ Come prima
 - ▶ A seconda dell'ordine in cui si verificano i valori di *set* e *res* si definisce la priorità tra i due

```
-- Flip flop di tipo D con set e
-- reset async
library ieee;
use ieee.std_logic_1164.all;

entity D_FF is
    port (d, clk, set, res : in std_logic;
          q : out std_logic);
end entity D_FF;

architecture behavioral of D_FF is
begin
    process (clk, set, res) is
    begin
        if set = '0' then
            q <= '1';
        elsif res = '0' then
            q <= '0';
        elsif rising_edge(clk) then
            q <= d;
        end if;
    end process;
end architecture behavioral;
```

Registri edge triggered

- ▶ **Flip flop di tipo D con reset sincrono**
 - ▶ Questa volta il reset è operativo solo al fronte del clock
 - ▶ Ne controlliamo quindi il valore solo dopo aver verificato la presenza di un fronte
 - ▶ Notare che il processo non è sensibile al segnale *res*
 - ▶ Che succederebbe se lo mettessimo lo stesso nella sensitivity list?

```
-- Flip flop di tipo D con reset sync
library ieee;
use ieee.std_logic_1164.all;

entity D_FF is
    port (d, clk, res : in std_logic;
          q : out std_logic);
end entity D_FF;

architecture behavioral of D_FF is
begin
    process (clk) is
    begin
        if rising_edge(clk) then
            if res = '0' then
                q <= '0';
            else
                q <= d;
            end if;
        end if;
    end process;
end architecture behavioral;
```

Registri edge triggered con load enable

► Flip flop di tipo D con controllo di caricamento

- L'uscita deve essere assegnata solo se il segnale *load* è attivo
- Il segnale *load* è sincrono, quindi non è necessario metterlo in sensitivity list
- Basta verificarne il valore prima di fare l'assegnazione
- Se *load* non è attivo non si fa nulla, quindi si mantiene il valore precedente

```
-- Flip flop di tipo D con load enable
library ieee;
use ieee.std_logic_1164.all;

entity D_FF is
    port (d, clk, load : in std_logic;
          q : out std_logic);
end entity D_FF;

architecture behavioral of D_FF is
begin
    process (clk) is
    begin
        if rising_edge(clk) then
            if load = '1' then
                q <= d;
            end if;
        end if;
    end process;
end architecture behavioral;
```

Registro a più bit

- ▶ **Esattamente come un registro normale**
 - ▶ Solo che l'ingresso e l'uscita di dati sono dei vettori
 - ▶ Possiamo parametrizzare la dimensione con un generic
 - ▶ Notare come si assegna 0 a q senza saperne la dimensione
 - ▶ Per il resto non cambia nulla
 - ▶ Facile aggiungere anche il load enable, etc.

```
-- Registro multibit
library ieee;
use ieee.std_logic_1164.all;

entity reg is
    generic (n : natural := 4);
    port (d : in std_logic_vector(n-1 downto 0);
          clk, res : in std_logic;
          q : out std_logic_vector(n-1 downto 0));
end entity reg;

architecture behavioral of reg is
begin
    process (clk, res) is
    begin
        if res = '0' then
            q <= (others => '0');
        elsif rising_edge(clk) then
            q <= d;
        end if;
    end process;
end architecture behavioral;
```

Registro a più bit

► Registro con

- Reset asincrono attivo basso
- Init sincrono attivo alto
- Load enable attivo alto

```
-- Registro multibit
library ieee;
use ieee.std_logic_1164.all;

entity reg is
    generic (n : natural := 4);
    port (d : in std_logic_vector(n-1 downto 0);
          clk, res, init, load : in std_logic;
          q : out std_logic_vector(n-1 downto 0));
end entity reg;

architecture behavioral of reg is
begin
    process (clk, res) is begin
        if res = '0' then
            q <= (others => '0');
        elsif rising_edge(clk) then
            if init = '1' then
                q <= (others => '0');
            elsif load = '1' then
                q <= d;
            end if;
        end if;
    end process;
end architecture behavioral;
```


Registro a scorrimento

► Ingresso seriale ed uscita seriale

- Viene anche chiamato Serial In Serial Out (SISO)
- Abbiamo bisogno di un segnale interno per memorizzare i dati
- Al fronte del clock si shiftano i bit
- Notare che l'ordine con cui si assegna non ha importanza
- Un'equazione al di fuori del processo assegna il valore di q
 - Si poteva mettere dentro al processo?
 - Se si, come?

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;

entity siso is
    port (d : in std_logic;
          clk : in std_logic;
          q : out std_logic);
end entity siso;

architecture behavioral of siso is
    signal reg : std_logic_vector(3 downto 0);
begin
    process (clk) is
    begin
        if rising_edge(clk) then
            reg(3) <= reg(2);
            reg(2) <= reg(1);
            reg(1) <= reg(0);
            reg(0) <= d;
        end if;
    end process;

    q <= reg(3);

end architecture behavioral;
```

Registro a scorrimento

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;

entity siso is
  port (d : in std_logic;
        clk : in std_logic;
        q : out std_logic);
end entity siso;

architecture behavioral of siso is
  signal reg : std_logic_vector(3 downto 0);
begin
  process (clk) is
  begin
    if rising_edge(clk) then
      reg(3) <= reg(2);
      reg(2) <= reg(1);
      reg(1) <= reg(0);
      reg(0) <= d;
      q <= reg(2);
    end if;
  end process;

end architecture behavioral;
```

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;

entity siso is
  port (d : in std_logic;
        clk : in std_logic;
        q : out std_logic);
end entity siso;

architecture behavioral of siso is
  signal reg : std_logic_vector(2 downto 0);
begin
  process (clk) is
  begin
    if rising_edge(clk) then
      q <= reg(2);
      reg(2) <= reg(1);
      reg(1) <= reg(0);
      reg(0) <= d;
    end if;
  end process;

end architecture behavioral;
```

Registro a scorrimento

► Ingresso seriale ed uscita parallela

- Viene anche chiamato Serial In Parallel Out (SIPO)
- Al fronte del clock si assegnano a q i bit meno significativi di q , tranne il più significativo, concatenati con d
- L'operatore di concatenazione è &

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;

entity sipo is
  generic (n : natural := 8);
  port (d : in std_logic;
        clk, res : in std_logic;
        q : buffer std_logic_vector(n-1 downto 0));
end entity sipo;

architecture behavioral of sipo is
begin
  process (clk, res) is
  begin
    if res = '0' then
      q <= (others => '0');
    elsif rising_edge(clk) then
      q <= q(n-2 downto 0) & d;
    end if;
  end process;
end architecture behavioral;
```

```
-- Per shiftare dall'altra parte
    elsif rising_edge(clk) then
      q <= d & q(n-1 downto 1);
```

Operatori aritmetici

- ▶ **La grande forza dei linguaggi è la loro capacità di astrarre operatori complessi**
 - ▶ Operatori aritmetici come somme, prodotti, etc.
 - ▶ Operatori di confronto
 - ▶ Per l'utente è sufficiente usarli, le tecniche di implementazione sono realizzate tramite strumenti automatici (ma è comunque utile conoscerle!)
- ▶ **Il VHDL nasce senza supporto nativo di questi operatori**
 - ▶ Sono disponibili per i tipi integer, che però non sono facilmente sintetizzabili
 - ▶ I `bit_vector` e gli `std_logic_vector` però sono solo insiemi di bit, e non sono interpretati come numeri
 - ▶ Vengono in aiuto una serie di librerie standard
 - ▶ Purtroppo vi sono vari modi di fare le stesse cose

Libreria numeric_std

- ▶ Introduce i tipi **signed** e **unsigned**
 - ▶ Sono dei vettori come gli std_logic_vector
 - ▶ La libreria definisce operatori aritmetici e di confronto (in modo differente per signed e unsigned)
 - ▶ Consente la trasformazione da e per std_logic_vector
- ▶ La libreria std_logic_arith è simile ma non è uno standard IEEE

```
-- Uso di operatori di confronto
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity cp is
  port (s : in std_logic_vector (7 downto 0);
        s_big, s_neg : out std_logic);
end entity cp;

architecture beh of cp is
begin
  s_big <= '1' when ( unsigned(s) > 200 ) else '0';
  s_neg <= '1' when ( signed(s) < 0 ) else '0';
end architecture beh;
```

Sommatore

- ▶ **Eseguiamo l'operazione come signed o unsigned**
 - ▶ Definiamo dei segnali temporanei di tipo signed o unsigned
 - ▶ Allineiamo a n+1 bit tutti i vettori
 - ▶ Per unsigned estendiamo con '0'
 - ▶ Per signed estendiamo con il valore del segno
 - ▶ Quindi si esegue la somma e si estrae il carry finale

```
-- Uso di operatori aritmetici
library ieee;
use ieee.std_logic_1164.all, ieee.numeric_std.all;

entity Adder is
    generic (n : natural := 4);
    port (a, b : in std_logic_vector (n-1 downto 0);
          cin : in std_logic;
          sum : out std_logic_vector (n-1 downto 0);
          cout : out std_logic);
end entity Adder;

architecture unsqnd of Adder is
    signal result, carry : unsigned(n downto 0);
    constant zeros : unsigned(n-1 downto 0) := (others => '0');
begin
    carry <= (zeros & cin);
    result <= ('0' & unsigned(a)) + ('0' & unsigned(b)) + carry;
    sum <= std_logic_vector( result(n-1 downto 0) );
    cout <= result(n);
end architecture unsqnd;

architecture sqnd of Adder is
    signal result, carry : signed(n downto 0);
    constant zeros : signed(n-1 downto 0) := (others => '0');
begin
    carry <= (zeros & cin);
    result <= (a(n-1) & signed(a)) + (b(n-1) & signed(b)) + carry;
    sum <= std_logic_vector( result(n-1 downto 0) );
    cout <= result(n);
end architecture sqnd;
```

Libreria std_logic_signed/unsigned

- Permette di usare gli std_logic direttamente come fossero dei numeri

```
-- Uso di operatori aritmetici
library ieee;
use ieee.std_logic_1164.all, ieee.std_logic_signed.all;

entity Adder is
  generic (n : natural := 4);
  port (a, b : in std_logic_vector(n-1 downto 0);
        sum : out std_logic_vector(n-1 downto 0);
        v : out std_logic);
end entity Adder;

architecture sgnd of Adder is begin
  process (a, b) is
    variable y : std_logic_vector(n-1 downto 0);
  begin
    y := a + b;
    if (a > 0 and b > 0 and y < 0) or
       (a < 0 and b < 0 and y > 0) then
      v <= '1';
    else
      v <= '0';
    end if;
    sum <= y;
  end process;
end architecture sgnd;
```

Contatore binario

- ▶ Realizzato sommando o sottraendo il valore numerico 1
 - ▶ Reset attivo basso
 - ▶ Funzionamento sincrono
 - ▶ Comando che definisce la direzione di conteggio

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity conta16 is
    port (res, clk, dir : in std_logic;
          y : buffer std_logic_vector(3 downto 0));
end entity conta16;

architecture behavioral of conta16 is
begin
    process (res, clk) is
    begin
        if (res = '0') then
            y <= (others => '0');
        elsif rising_edge(clk) then
            if (dir = '1') then
                y <= y + 1;
            else
                y <= y - 1;
            end if;
        end if;
    end process;
end architecture behavioral;
```


Contatore binario

► Contatore con

- Reset attivo basso
- Comando di count enable
- Uscita di terminal count

► Nota

- Il conteggio è solo un segnale interno, definito direttamente come tipo numerico
- Il terminal count è determinato in modo combinatorio fuori dal processo tramite una equazione

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity conta256 is
    port (res, clk, count_en : in std_logic;
          tc : out std_logic);
end entity conta256;

architecture behavioral of conta256 is

    signal y : signed(7 downto 0);

begin

    process (res, clk) is
    begin
        if res = '0' then
            y <= (others => '0');
        elsif rising_edge(clk) then
            if count_en = '1' then
                y <= y + 1;
            end if;
        end if;
    end process;

    tc <= '1' when y = 255 else '0';

end architecture behavioral;
```

Contatore binario

- ▶ **Terminal count nel processo**
 - ▶ Vantaggio: tc è ora l'uscita di un flip flop
 - ▶ Questo codice **non funziona** come si vorrebbe
 - ▶ Il segnale y non viene aggiornato subito, ma solo a fine processo, quindi tc non ha il valore corretto

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity conta256 is
    port (res, clk, count_en : in std_logic;
          tc : out std_logic);
end entity conta256;

architecture behavioral of conta256 is
    signal y : unsigned(7 downto 0);
begin

    process (res, clk) is
    begin
        if res = '0' then
            y <= (others => '0'); tc <= '0';
        elsif rising_edge(clk) then
            if count_en = '1' then
                y <= y + 1;
            end if;
            if y = 255 then tc <= '1';
            else tc <= '0';
            end if;
        end if;
    end process;

end architecture behavioral;
```

Contatore binario

- ▶ **Terminal count nel processo**
 - ▶ Risolvibile tramite una variabile

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity conta256 is
    port (res, clk, count_en : in std_logic;
          tc : out std_logic);
end entity conta256;

architecture behavioral of conta256 is
begin

    process (res, clk) is
        variable y : unsigned(7 downto 0);
    begin
        if res = '0' then
            y := (others => '0'); tc <= '0';
        elsif rising_edge(clk) then
            if count_en = '1' then
                y := y + 1;
            end if;
            if y = 255 then tc <= '1';
            else tc <= '0';
            end if;
        end if;
    end process;

end architecture behavioral;
```

Contatore binario

- ▶ **Terminal count nel processo**
 - ▶ Oppure guardando lo stato futuro invece di quello presente
 - ▶ **if y = 254 then**
 - ▶ **if y + 1 = 255 then**
 - ▶ Importante verificare che anche count_en sia attivo!

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity conta256 is
    port (res, clk, count_en : in std_logic;
          tc : out std_logic);
end entity conta256;

architecture behavioral of conta256 is
    signal y : unsigned(7 downto 0);
begin

    process (res, clk) is
    begin
        if res = '0' then
            y <= (others => '0'); tc <= '0';
        elsif rising_edge(clk) then
            if count_en = '1' then
                y <= y + 1;
                if y = 254 then tc <= '1';
                else tc <= '0';
                end if;
            end if;
        end if;
    end process;

end architecture behavioral;
```

Contatore binario

► Terminal count nel processo

- Come prima ma fuori dal controllo di count_en (funzionamento errato!)
- Rischioso, se count_en non è attivo, il tc può essere attivo quando y = 254
- Inoltre il tc rimane attivo un solo ciclo di clock, anche se y rimane a 255

```
-- Registro a scorrimento
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity conta256 is
    port (res, clk, count_en : in std_logic;
          tc : out std_logic);
end entity conta256;

architecture behavioral of conta256 is
    signal y : unsigned(7 downto 0);
begin

    process (res, clk) is
    begin
        if res = '0' then
            y <= (others => '0'); tc <= '0';
        elsif rising_edge(clk) then
            if count_en = '1' then
                y <= y + 1;
            end if;
            if y = 254 then tc <= '1';
            else tc <= '0';
            end if;
        end if;
    end process;

end architecture behavioral;
```

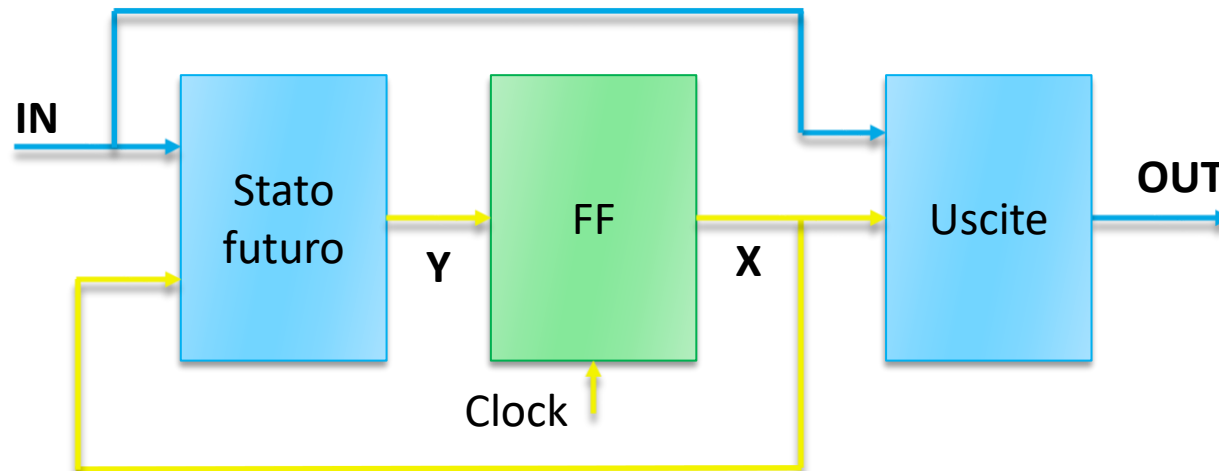
- ▶ **Il VHDL offre molte potenzialità**
 - ▶ Ne abbiamo viste per ora solo alcune
 - ▶ Ci forniscono comunque i blocchi base con cui sviluppare sistemi più complessi
- ▶ **Simulazione e sintesi**
 - ▶ Si scrive il modello del sistema in modo che possa essere sintetizzato, oltre ad essere simulato
 - ▶ Il modello del testbench e/o di oggetti già realizzati possono essere scritti solo per la simulazione
- ▶ **Data flow e comportamentale**
 - ▶ Due stili per rappresentare funzioni booleane combinatorie e sequenziali
 - ▶ Da scegliere a seconda della convenienza
 - ▶ Attenzione a garantire il comportamento combinatorio e sequenziale verificando gli assegnamenti lungo tutti i flussi di esecuzione

Macchine a stati in VHDL

Come rappresentare un digramma in testo

Macchine a stati

- ▶ **Le realizziamo seguendo la normale separazione tra parte sequenziale e combinatoria**
 - ▶ La parte sequenziale sarà sostanzialmente simile ad un registro a molti bit
 - ▶ La parte combinatoria può essere spezzata in due, per il calcolo dello stato futuro e delle uscite
 - ▶ Ogni blocco può essere rappresentato da un processo all'interno dell'architettura



Esempio del derivatore

► Parte sequenziale

- Un segnale interno per lo stato presente ed uno per lo stato futuro
- Stato come tipo enumerato, per non decidere subito la codifica
- Al colpo di clock lo stato futuro diventa presente

```
-- Derivatore
library ieee; use ieee.std_logic_1164.all;

entity derivatore is
  port (s, clk, res : in std_logic;
        y : out std_logic);
end entity derivatore;

architecture behavioral of derivatore is
  type stato is (a, b, c, d);
  signal present_state, next_state : stato;
begin
  seq: process (res, clk) is
  begin
    if res = '0' then
      present_state <= a;
    elsif rising_edge(clk) then
      present_state <= next_state;
    end if;
  end process seq;
-- Continua alla slide successiva
```

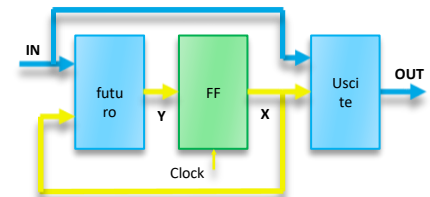
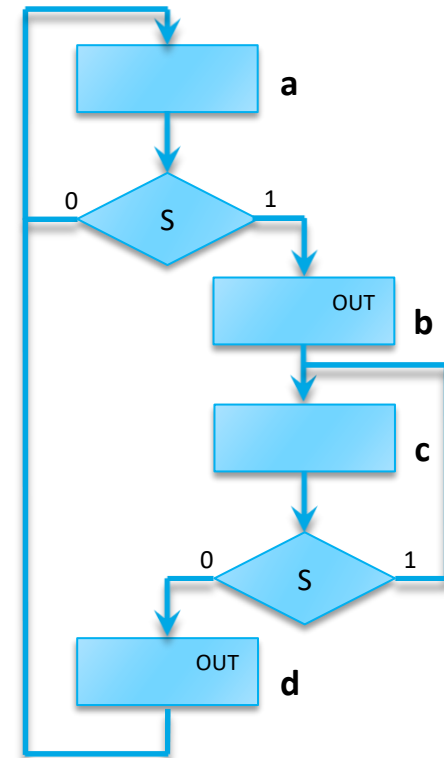
Definizione del tipo di stato

Segnali di stato

Processo per la parte sequenziale, sensibile a reset e clock

Stato iniziale

Aggiornamento stato



Processi combinatori

-- Continua dalla slide precedente

```

futuro: process (present_state, s) is
begin
  case present_state is
    when a =>
      if s = '0' then
        next_state <= a;
      else
        next_state <= b;
      end if;
    when b =>
      next_state <= c;
    when c =>
      if s = '0' then
        next_state <= d;
      else
        next_state <= c;
      end if;
    when d =>
      next_state <= a;
    end case;
  end process futuro;

  uscite: process (present_state) is
  begin
    y <= '0';
    if present_state = b or present_state = d then
      y <= '1';
    end if;
  end process uscite;
end architecture behavioral;
  
```

Stato futuro dipende da quello presente e dall'ingresso

Costrutto **case** per i processi (non si può usare quello data flow)

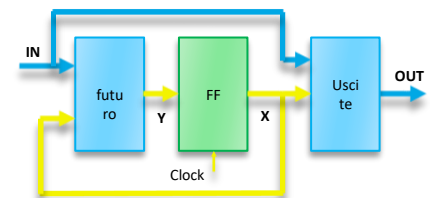
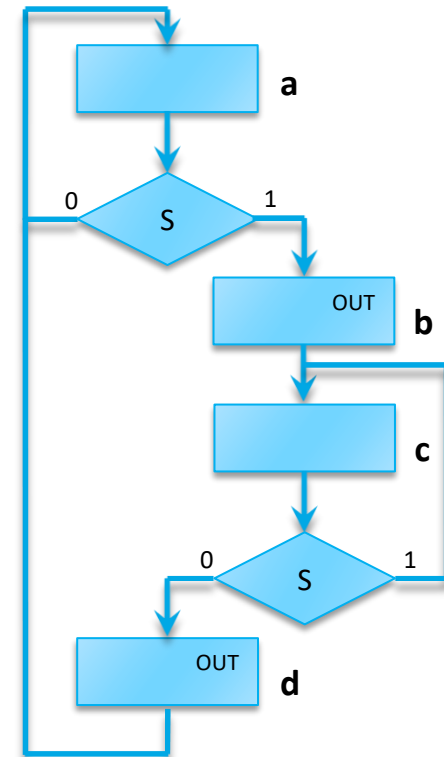
Un caso per ogni stato

Stato futuro dipende dall'ingresso

Si assegna qui, diventerà presente al colpo di clock

Macchina di Moore, uscite dipendono solo dallo stato

Uscita a 1 negli stati **b** e **d**



Osservazioni

- ▶ **Solo il present_state diventa un insieme di flip flop, mentre il next_state no**
 - ▶ Il present_state è pilotato da un processo sequenziale
 - ▶ Ci sono dei casi in cui viene attivato il processo e non gli viene assegnato alcun valore
 - ▶ Il next_state è pilotato da un processo combinatorio
 - ▶ Per tutti i casi possibili si assegna un valore, quindi non genera un elemento di memoria
 - ▶ Le uscite sono anche pilotate da un processo combinatorio, quindi ovviamente non generano un elemento di memoria
- ▶ **Si possono mettere stato futuro e uscite insieme**
 - ▶ Si fa un solo processo che pilota entrambi
 - ▶ Riduce la quantità di codice da scrivere

Derivatore come macchina di Mealy

► Gli stati ora sono solo due

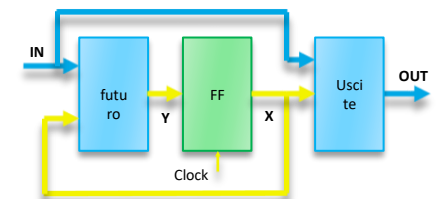
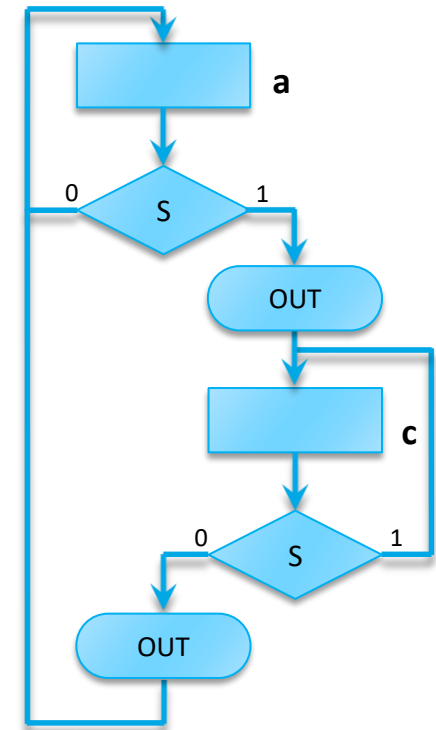
- Ma la parte sequenziale sostanzialmente non cambia
- Infatti è praticamente sempre uguale per tutte le macchine a stati!

```
-- Derivatore
library ieee; use ieee.std_logic_1164.all;

entity derivatore is
  port (s, clk, res : in std_logic;
        y : out std_logic);
end entity derivatore;

architecture behavioral of derivatore is
  type stato is (a, c);
  signal present_state, next_state : stato;
begin
  seq: process (res, clk) is
  begin
    if res = '0' then
      present_state <= a;
    elsif rising_edge(clk) then
      present_state <= next_state;
    end if;
  end process seq;
-- Continua alla slide successiva
```

Definizione del tipo di stato



Processi combinatori

-- Continua dalla slide precedente

```
futuro: process (present_state, s) is
begin
  next_state <= present_state;
  case present_state is
    when a =>
      if s = '1' then
        next_state <= c;
      end if;
    when c =>
      if s = '0' then
        next_state <= a;
      end if;
  end case;
end process futuro;

uscite: process (present_state, s) is
begin
  y <= '0';
  if (present_state = a and s = '1') or
     (present_state = c and s = '0') then
    y <= '1';
  end if;
end process uscite;

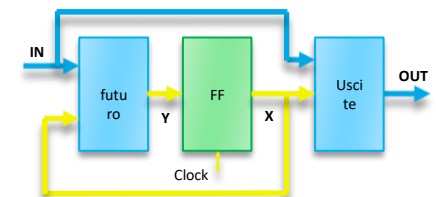
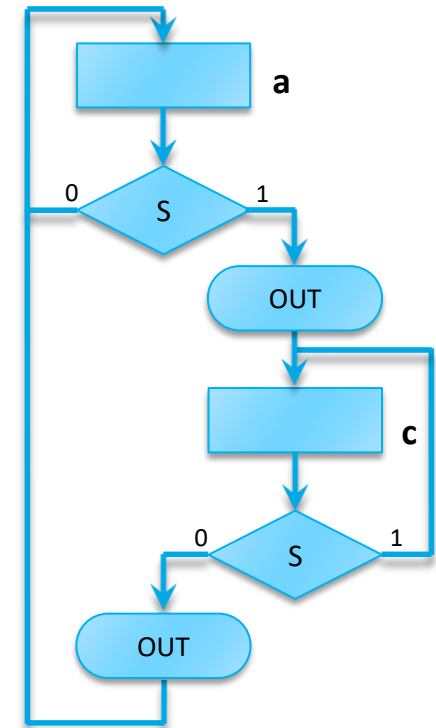
end architecture behavioral;
```

Stato futuro dipende da quello presente e dall'ingresso

Utile default: se non diciamo altro stiamo nello stesso stato

Macchina di Mealy, uscite dipendono dallo stato e dagli ingressi

Uscita a 1 nello stato a se s=1 e nello stato c se s=0



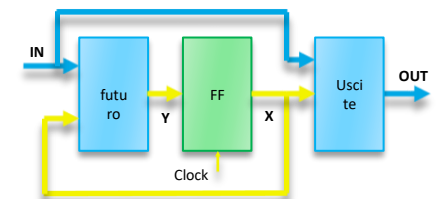
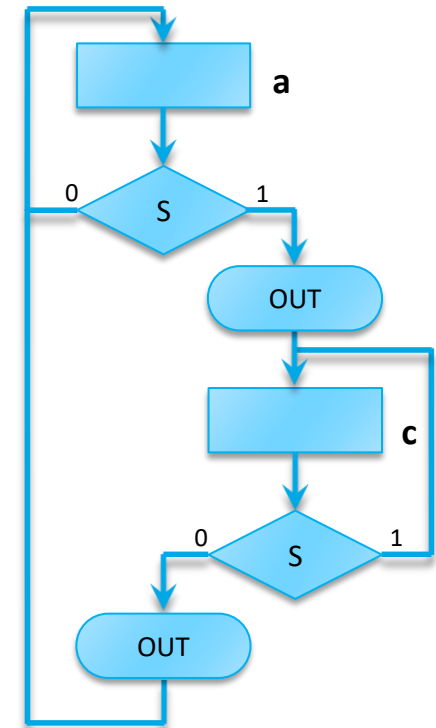
Un solo processo combinatorio

-- Continua dalla slide precedente

```
comb: process (present_state, s) is
begin
  next_state <= present_state;
  y <= '0';
  if (present_state = a and s = '1') then
    next_state <= c;
    y <= '1';
  elsif (present_state = c and s = '0') then
    next_state <= a;
    y <= '1';
  end if;
end process comb;

end architecture behavioral;
```

Pilota sia lo stato futuro, sia le uscite



Un solo processo sequenziale/combinatorio

```
-- Derivatore
library ieee; use ieee.std_logic_1164.all;

entity derivatore is
  port (s, clk, res : in std_logic;
        y : out std_logic);
end entity derivatore;

architecture behavioral of derivatore is
  type stato is (a, c);
  signal present_state : stato;
begin
  seq: process (res, clk) is
  begin
    if res = '0' then
      present_state <= a;
    elsif rising_edge(clk) then
      case present_state is
        when a => if s = '1' then
                    present_state <= c;
                  end if;
        when c => if s = '0' then
                    present_state <= a;
                  end if;
      end case;
    end process seq;

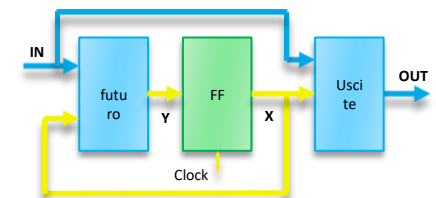
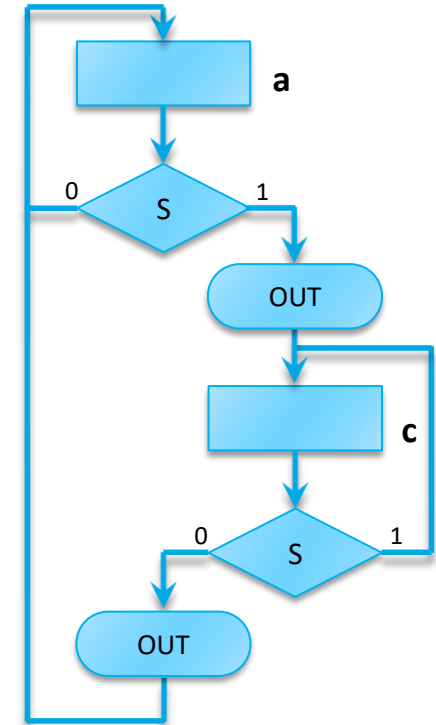
    y <= '1' when (present_state = a and s = '1') or
                  (present_state = c and s = '0')
              else '0';
  end architecture behavioral;
```

Stato futuro implicito, viene comunque sintetizzato

...ma assegnato solo al fronte del clock

present_state calcolato in termini di se stesso...

uscite calcolate come equazione in questo caso



Codifica dello stato

- ▶ **Abbiamo lasciato lo stato in forma simbolica**
 - ▶ Questo ci consente di verificare la funzionalità indipendentemente dalla codifica
 - ▶ Alcuni strumenti consentono di effettuare una codifica automatica
- ▶ **E' possibile specificare la codifica in forma esplicita**
 - ▶ Dipende dai tool usati
 - ▶ Si usano attributi

```
attribute enum_encoding : string;  
type stato is (a, b, c, d);  
attribute enum_encoding of stato : type is "00 01 11 10";
```

- ▶ **Oppure si usano costanti esplicite per dare valori agli elementi della enumerazione**
 - ▶ Indipendente dal tool

```
-- Derivatore  
architecture behavioral of derivatore is  
  type stato is std_logic_vector(1 downto 0);  
  signal present_state, next_state : stato;  
  constant a : stato := "00";  
  constant b : stato := "01";  
  constant c : stato := "11";  
  constant d : stato := "10";  
begin  
  seq: process (res, clk) is  
  begin  
    if res = '0' then  
      present_state <= a;  
    elseif rising_edge(clk) then  
      present_state <= next_state;  
    end if;  
  end process seq;  
  
  futuro: process (present_state, s) is  
  begin  
    case present_state is  
      when a => if s = '0' then  
        next_state <= a;  
      else  
        next_state <= b;  
      end if;  
      when b => next_state <= c;  
      when c => if s = '0' then  
        next_state <= d;  
      else  
        next_state <= c;  
    end case;  
  end process futuro;  
end architecture derivatore;
```


Codifica one-hot

► Facile cambiare la codifica

- Basta modificare la definizione delle costanti e del tipo
- Occorre però fare attenzione alle possibili alternative per lo stato presente
- Per la simulazione non ci sono problemi
- Ma il sintetizzatore pensa che `present_state` possa essere per esempio 0110
- Non trovando un ramo di controllo per quel valore, aggiunge un latch spurio per ricordare il valore precedente
- Meglio allora mettere un caso di default al **case**

```
-- Derivatore
architecture behavioral of derivatore is
    type stato is std_logic_vector(3 downto 0);
    signal present_state, next_state : stato;
    constant a : stato := "0001";
    constant b : stato := "0010";
    constant c : stato := "0100";
    constant d : stato := "1000";
begin
    seq: process (res, clk) is
    begin
        ...
    futuro: process (present_state, s) is
    begin
        case present_state is
            when a =>
                if s = '0' then next_state <= a;
                else next_state <= b;
                end if;
            when b =>
                next_state <= c;
            when c =>
                if s = '0' then next_state <= d;
                else next_state <= c;
                end if;
            when d =>
                next_state <= a;
            when others =>
                next_state <= "XXXX";
            end case;
        end process futuro;
        ...
    end
```

Testbench per circuiti sequenziali

- ▶ **Abbiamo già visto come creare testbench tramite processi**
 - ▶ Per i circuiti sequenziali occorre generare anche i segnali di clock e di reset
 - ▶ Il processo del clock non si ferma mai
 - ▶ Quello per il reset termina con l'ultimo **wait**
 - ▶ Nel resto del codice occorre quindi definire i restanti segnali di ingresso dell'oggetto da testare
 - ▶ Ed occorre istanziare l'oggetto da testare

```
-- Testbench sequenziale
library ieee; use ieee.std_logic_1164.all;

entity test_seq is
end entity test_seq;

architecture behavioral of test_seq is
    signal segnali-interni;
begin
    clk: process is
    begin
        clock <= '0';
        wait for 5 ns;
        clock <= '1';
        wait for 5 ns;
    end process clk;

    rst: process is
    begin
        reset <= '1';
        wait for 5 ns;
        reset <= '0';
        wait for 50 ns;
        reset <= '1';
        wait;
    end process rst;

-- Altri segnali

-- Istanziamento entità under test
```

Memorie

- ▶ **Le memorie sono incluse in buona parte dei circuiti digitali**
 - ▶ ROM: Read Only Memory
 - ▶ RAM: Random Access Memory
- ▶ **E' utile avere modelli di memorie**
 - ▶ Soprattutto per la simulazione
 - ▶ Normalmente non vengono sintetizzate (verrebbero troppo grosse) ma realizzate a mano o tramite generatori automatici
- ▶ **Funzionamento**
 - ▶ Ad ogni indirizzo corrisponde un valore
 - ▶ In pratica si tratta di un array

Modello per una ROM

- ▶ **Scriviamo i valori predefiniti in un array**
 - ▶ Per chiarezza ne definiamo il tipo
 - ▶ Usiamo un integer per indicizzare l'array (tanto non va sintetizzata)
 - ▶ Possibile specificare un tempo di ritardo
- ▶ **E' come fare una tabella della verità**
 - ▶ Possiamo usare una ROM per implementare una qualunque funzione combinatoria
 - ▶ Una EEPROM spesso usata per questo scopo

```
-- Modello di una ROM
library ieee;
use ieee.std_logic_1164.all;

entity ROM is
    port (address : in integer range (0 to 15);
          data : out std_logic_vector (7 downto 0));
end entity ROM;

architecture dataflow of ROM is
    type rom_array is array (0 to 15) of
        std_logic_vector(7 downto 0);
    constant rom : rom_array := ( "01010101",
                                    "10101010",
                                    "00000111",
                                    "01011101",
                                    "01110101",
                                    ...,
                                    "11111111",
                                    "00110111" );

begin
    data <= rom(address) after 10 ns;
end architecture dataflow;
```

Modello di una RAM statica

- ▶ **I dati possono essere letti e scritti**
 - ▶ Simile alla ROM
 - ▶ Usiamo il bus dati per leggere e scrivere, quindi **inout**
 - ▶ Tre segnali di controllo attivi bassi
 - ▶ **cs**: chip select per selezionare la memoria
 - ▶ **oe**: output enable per leggere
 - ▶ **we**: write enable per scrivere
- ▶ **Usiamo un processo sequenziale**
 - ▶ Di default le uscite sono in alta impedenza
 - ▶ Vengono assegnate un valore solo se **cs** e **oe** sono entrambi attivi
 - ▶ Quando si scrive, il valore di *data* viene assegnato da qualche altro componente
 - ▶ La funzione `resolve` fa override dell'assegnazione a Z

```
-- Modello di una RAM statica
library ieee;
use ieee.std_logic_1164.all;

entity RAM16x8 is
    port (address : in integer range (0 to 15);
          data : inout std_logic_vector(7 downto 0)
          cs, oe, we : in std_logic);
end entity RAM16x8;

architecture behavioral of RAM16x8 is
begin
    process (address, cs, we, oe, data) is
        type ram_array is array (0 to 15) of
            std_logic_vector(7 downto 0);
        variable mem : ram_array;
    begin
        data <= (others => 'Z');
        if cs = '0' then
            if oe = '0' then
                data <= mem(address);
            elsif we = '0' then
                mem(address) := data;
            end if;
        end if;
    end process;
end architecture behavioral;
```

Funzioni

- ▶ **E' possibile, come in altri linguaggi, fattorizzare il codice**
 - ▶ Una funzione prende dei parametri in ingresso e restituisce un valore in uscita
 - ▶ Utile per definire essenzialmente nuovi operatori
 - ▶ Fa uso al suo interno degli stessi statement sequenziali usati in un processo
 - ▶ A differenza di un processo viene attivata alla chiamata, invece che a causa del cambiamento di un segnale
 - ▶ I parametri formali sono sempre ingressi, e non sono modificabili
 - ▶ Una funzione può usare variabili, costanti, altre funzioni, etc.
 - ▶ Per ritornare un valore si usa **return** (valore);

```
-- Sintassi funzione
function nome-funzione (
  nome-parametro : in tipo-parametro;
  . . . . .
  nome-parametro : in tipo-parametro)
  return tipo-risultato is
  dichiarazioni-di-tipo
  dichiarazioni-di-costanti
  dichiarazioni-di-variabili
  dichiarazioni-di-funzioni
begin
  statement-sequenziale;
  . . . . .
  statement-sequenziale;
end nome-funzione;
```

Parametri formali, sempre **in**,
non modificabili all'interno
della funzione

Tipo del valore di ritorno

Esempio di funzioni

► Definizione di una funzione add

```
-- Esempio funzione
library ieee; use ieee.std_logic_1164.all;

entity s is
  port (p1, q1 : in std_logic_vector (7 downto 0);
        p2, q2 : in std_logic_vector (3 downto 0);
        y1 : out std_logic_vector (7 downto 0);
        y2 : out std_logic_vector (3 downto 0));
end entity s;

architecture beh of s is
  function add (a, b : in std_logic_vector)
    return std_logic_vector is
    variable s : std_logic_vector(a'range);
    variable carry : std_logic;
  begin
    carry := '0';
    for i in a'low to a'high loop
      s(i) := a(i) xor b(i) xor carry;
      carry := (a(i) and b(i)) or
                (carry and a(i)) or
                (carry and b(i));
    end loop;
    return s;
  end function add;
begin
  y1 <= add(p1, q1);
  y2 <= add(p2, q2);
end architecture beh;
```

Usiamo variabili per fare una somma bit a bit iterativa

Instanziazione della funzione con parametri differenti

Procedure

- ▶ **Simili alle funzioni, ma possono ritornare più di un valore**
 - ▶ I parametri formali possono essere sia di tipo **in** sia di tipo **out**
 - ▶ I parametri formali possono essere segnali o variabili (di default sono variabili)
 - ▶ La classe (variabile o segnale) dei parametri effettivi deve essere la stessa di quelli formali
 - ▶ Per usare segnali
a, b : **in signal** std_logic_vector;

```
-- Esempio procedura
procedure somma (a, b : in std_logic_vector;
                y : out std_logic_vector;
                cout : out std_logic) is
    variable carry : std_logic;
begin
    carry := '0';
    for i in a'low to a'high loop
        y(i) := a(i) xor b(i) xor carry;
        carry := (a(i) and b(i)) or (carry and a(i))
                or (carry and b(i));
    end loop;
    cout := carry;
end procedure somma;
```

Ingressi

Valori di ritorno

Configurazioni

- ▶ **Una specifica VHDL è costituita da dichiarazioni di entità e di architetture**
 - ▶ Ogni volta che viene utilizzata una entità si può scegliere una specifica architettura
 - ▶ Le configurazioni consentono di definire quali architetture utilizzare per il sistema
 - ▶ Facile cambiare architetture scambiando le configurazioni
- ▶ **Accesso a file**
 - ▶ Come nei normali linguaggi di programmazione è possibile scrivere e leggere i file
 - ▶ Utile solo per la simulazione
 - ▶ Si possono usare file per leggere valori per i segnali di ingresso o per scriverci valori di uscita

- ▶ **Abbiamo gli strumenti per realizzare sistemi complessi**
 - ▶ Il VHDL ci consente di esprimere il design in maniera semplice
 - ▶ Ci solleva da un gran numero di considerazioni implementative
- ▶ **Fondamentale la metodologia**
 - ▶ Separazione tra funzione e tempistiche tramite progetto sincrono
 - ▶ Separazione tra parte sequenziale e combinatoria
 - ▶ Separazione tra data path e unità di controllo
 - ▶ Suddivisione in componenti e macchine correlate
- ▶ **Il VHDL ci supporta con metodi di verifica e sintesi**
 - ▶ La verifica è necessaria per individuare errori presto durante il ciclo di progetto
 - ▶ E' una fase spesso più onerosa del progetto stesso
 - ▶ Utile verificare bene i componenti individualmente, e poi concentrarsi solo sulla loro interazione
 - ▶ La sintesi fornisce direttamente il circuito con diverse modalità realizzative

Gestione delle uscite in VHDL

- ▶ **In VHDL, le uscite sono gestite da un apposito processo**
 - ▶ Abbiamo anche detto di non includerle nel processo sequenziale, altrimenti diventano dei registri
 - ▶ Ed entrano a far parte dello stato

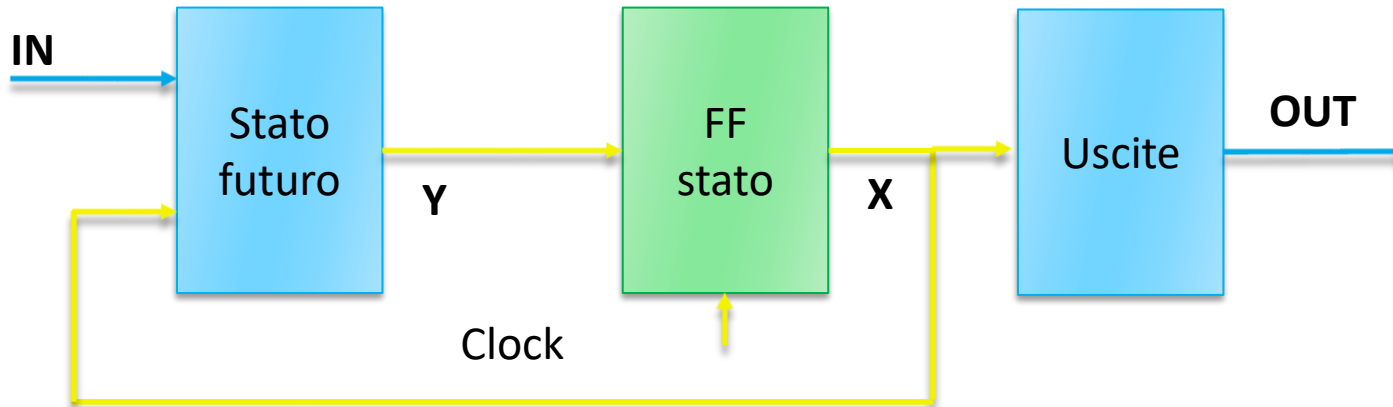
```
-- Processo per il calcolo delle uscite

uscite: process (present_state) is
begin
  case present_state is
    when b =>
      y <= '1';
    when d =>
      y <= '1';
    when others =>
      y <= '0';
  end case;
end process uscite;

end architecture behavioral;
```

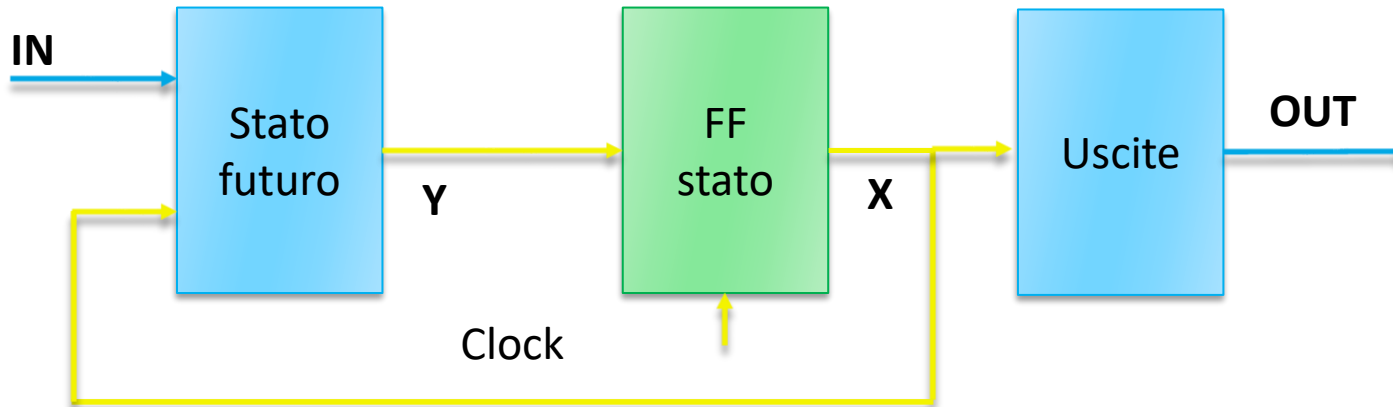
Gestione delle uscite

- ▶ In una macchina di Moore, le uscite dipendono solo dallo stato
 - ▶ Uscite calcolate con una rete combinatoria
 - ▶ La rete combinatoria potrebbe impiegare tempo a calcolare le uscite, che si stabilizzano durante il ciclo di clock
 - ▶ Le uscite potrebbero presentare dei glitch, magari indesiderati



Gestione delle uscite

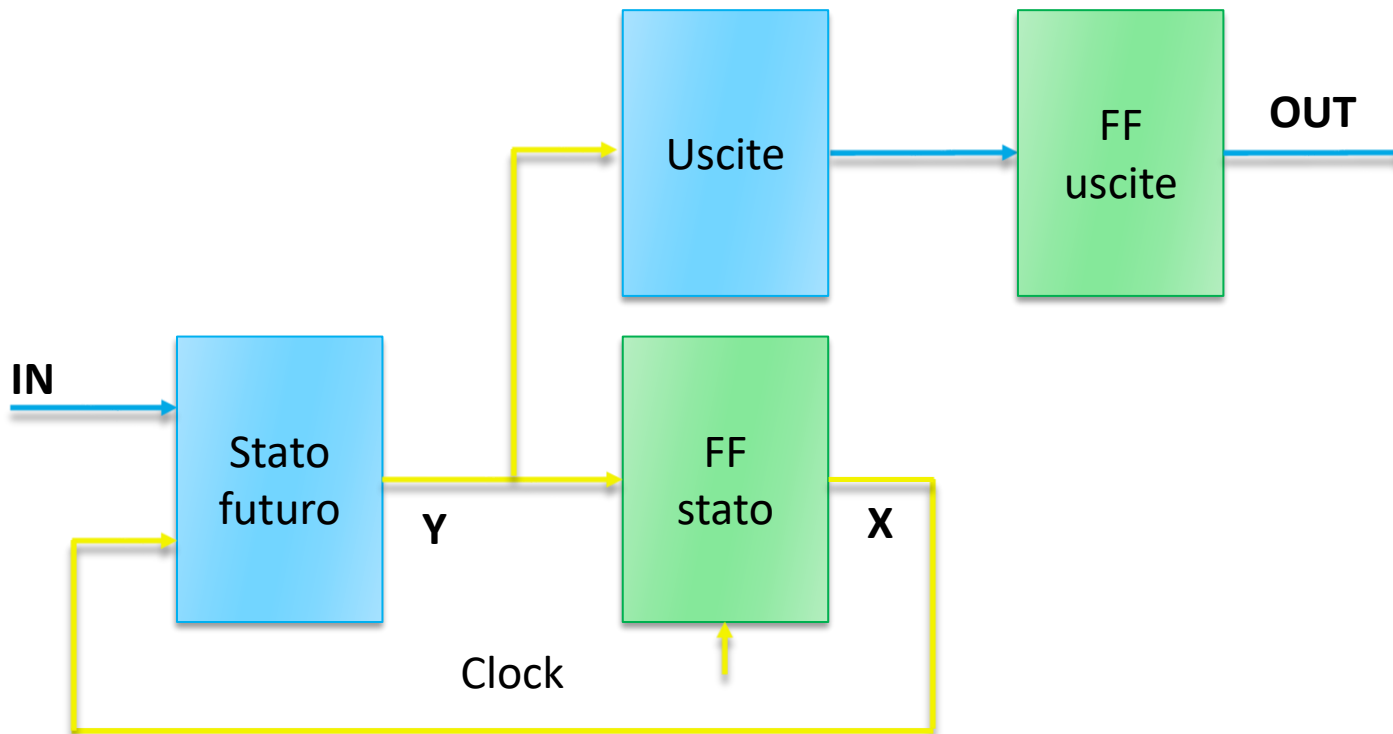
- ▶ Potrebbe essere utile fare in modo che le uscite corrispondano alle uscite di qualche flip flop
 - ▶ In questo modo si aggiornano subito dopo il fronte
 - ▶ E non hanno glitch
 - ▶ In certi casi occorre però aggiungere qualche variabile di stato
 - ▶ Occorre sapere il valore delle uscite *prima* dell'arrivo del fronte
 - ▶ Questo è semplice da fare nel caso delle macchine di Moore



Uscite registrate

► Anticipiamo il calcolo delle uscite

- Invece di usare lo stato presente, usiamo lo stato futuro, che conosciamo già il ciclo di clock prima
- A questo punto basta registrare le uscite con dei flip flop



Uscite registrate in VHDL

► Occorre gestirle come uno stato

- Ogni uscita avrà la versione presente (e.g., y) e la versione futura (e.g., next_y)
- Il calcolo delle uscite si basa sul next_state, e aggiorna next_y
- Un processo sequenziale aggiorna le uscite effettive

```
-- Calcolo uscite

next_uscite: process (next_state) is
begin
  case next_state is
    when b =>
      next_y <= '1';
    when d =>
      next_y <= '1';
    when others =>
      next_y <= '0';
  end case;
end process next_uscite;
```

```
-- Registrazione uscite

uscite: process (res, clock) is
begin
  if res = '0' then
    y <= '0';
  elsif rising_edge( clock ) then
    y <= next_y;
  end if;
end process uscite;
```

Uscite registrate in VHDL

- ▶ **Occorre gestirle come uno stato**
 - ▶ Volendo si può utilizzare un processo solo
 - ▶ Notate che al reset le uscite vanno messe al valore che assumono nello stato iniziale

```
-- Registrazione uscite

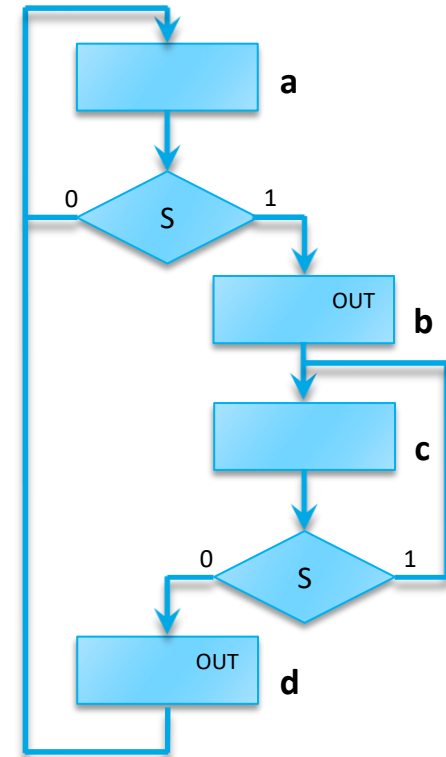
uscita: process (res, clock) is
begin
    if res = '0' then
        y <= '0';
    elsif rising_edge( clock ) then
        case next_state is
            when b =>
                y <= '1';
            when d =>
                y <= '1';
            when others =>
                y <= '0';
        end case;
    end if;
end process uscita;
```


Oppure si usa un singolo processo

```
-- Derivatore
library ieee; use ieee.std_logic_1164.all;

entity derivatore is
  port (s, clk, res : in std_logic;
        y : out std_logic);
end entity derivatore;

architecture behavioral of derivatore is
  type stato is (a, b, c, d);
  signal present_state : stato;
begin
  seq: process (res, clk) is
  begin
    if res = '0' then
      present_state <= a;
      y <= '0';
    elsif rising_edge(clk) then
      case present_state is
        when a => if s = '1' then
                     present_state <= b;
                     y <= '1';
                   end if;
        when b => present_state <= c; y <= '0';
        when c => if s = '0' then
                     present_state <= d;
                     y <= '1';
                   end if;
        when d => present_state <= a; y <= '0';
      end case;
    end if;
  end process seq;
```

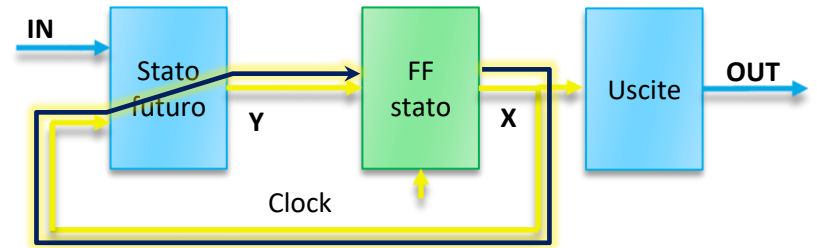


Soluzione molto utilizzata per
le macchine di Moore

Osservazioni

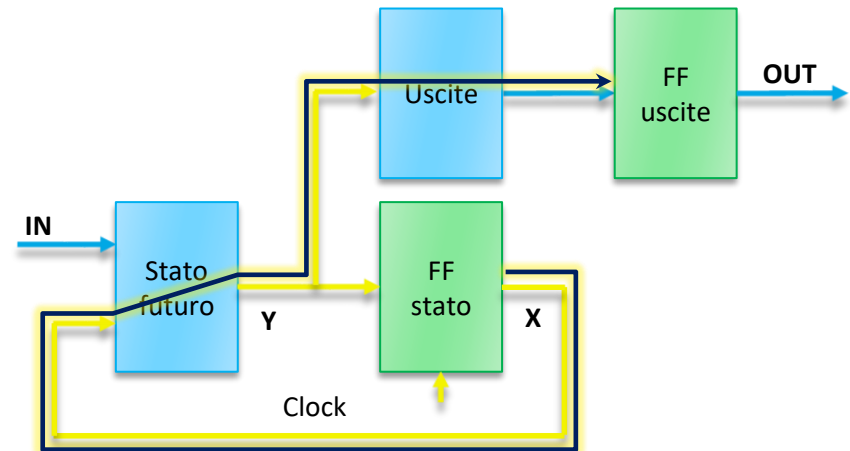
► Vantaggi

- Le uscite sono disponibili sin dall'inizio del ciclo di clock
- Le uscite rimangono stabili per tutto il ciclo di clock, e non hanno glitch



► Svantaggi

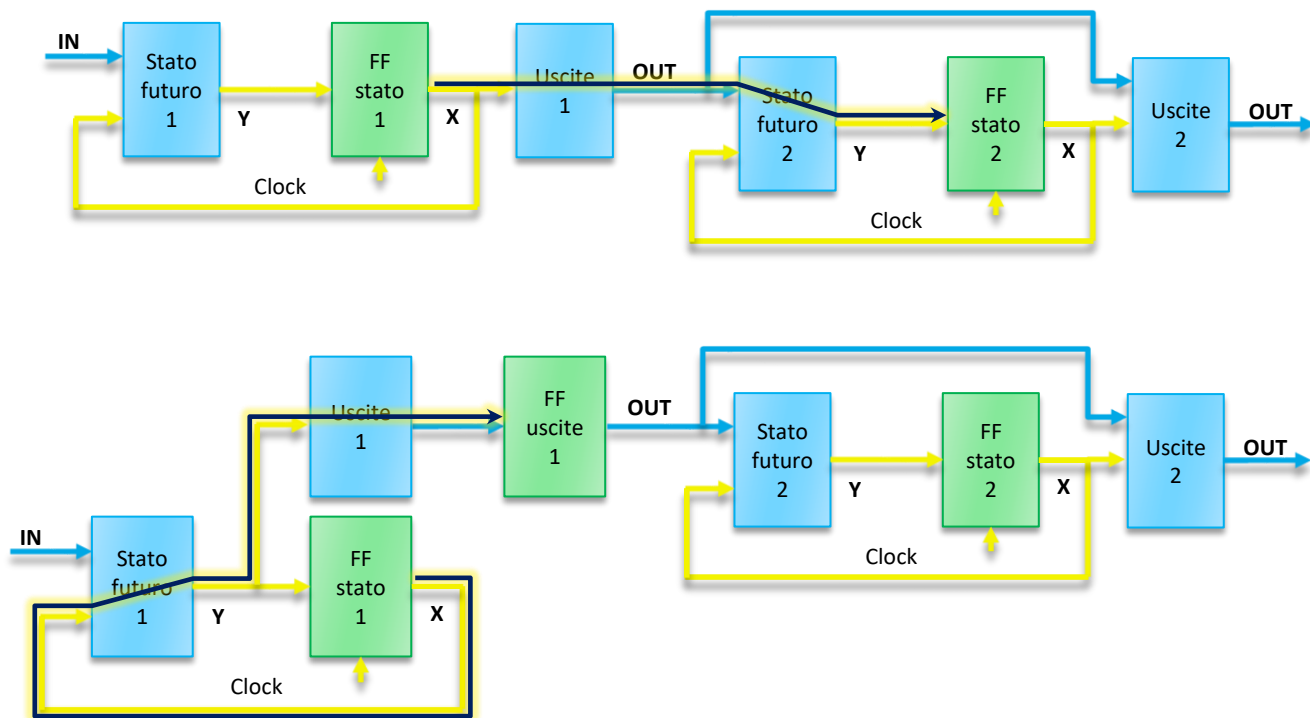
- Il circuito viene più grosso perché devo aggiungere dei FF
- Si rischia di dover allungare il ciclo di clock, perché attraverso due circuiti combinatori



Ritardi tra macchine comunicanti

► Quale è meglio?

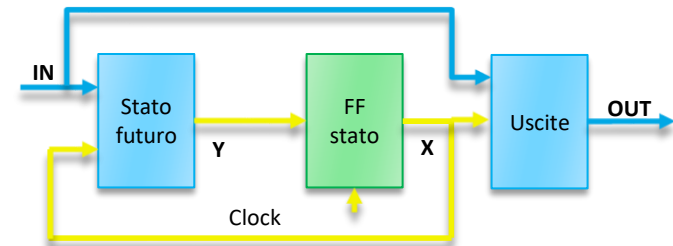
- Dipende dai valori dei ritardi dei vari blocchi combinatori



Macchine di Mealy

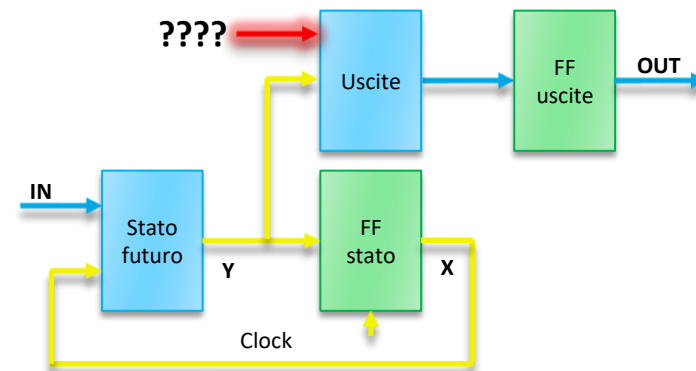
▶ La procedura funziona con una macchina di Moore

- ▶ Abbiamo spostato il blocco delle uscite da dopo i flip flop a prima dei flip flop
- ▶ Calcoliamo il valore delle uscite nel ciclo precedente usando lo stato futuro
- ▶ Poi lo ritardiamo con i FF per porlo in uscita nel ciclo giusto



▶ Non funziona per una di Mealy!!

- ▶ Per ogni ciclo, le uscite dipendono dal valore degli ingressi **nello stesso ciclo**
- ▶ Dovremmo sapere il valore degli ingressi con un ciclo di anticipo
- ▶ Ma questa informazione non è disponibile



Macchine a stati correlate

- ▶ **Un sistema complesso non può essere progettato come una unica entità**
 - ▶ Lo abbiamo già visto più volte
 - ▶ Una sola macchina a stati avrebbe troppi stati per poter essere gestita in modo sensato
- ▶ **Normalmente diverse macchine a stati interagiscono tra loro per fornire la funzionalità richiesta**
 - ▶ Le singole macchine a stati possono avere uno spazio degli stati molto più ristretto
 - ▶ Supponiamo di avere due macchine a stati ognuna, con n stati
 - ▶ Ognuna può trovarsi in uno qualunque dei suoi stati (non è esattamente vero...)
 - ▶ Quindi, complessivamente, il sistema avrà n^2 stati
 - ▶ Il partizionamento ci permette di trattare solo $2n$ stati invece di n^2
 - ▶ Occorre però tener conto dell'interazione

Esempio

- ▶ **Supponiamo di voler progettare un sistema che sia in grado di ricevere ed eseguire vari comandi**
 - ▶ Un comando causa la trasmissione di un dato lungo una linea seriale
 - ▶ Un altro permette di calcolare una media mobile dei dati ricevuti
 - ▶ E così via
- ▶ **I comandi sono forniti su una linea seriale**
 - ▶ La linea è al valore 0 quando in idle, mentre un 1 introduce un comando (bit di start)
 - ▶ Per esempio la sequenza 1101 indica il comando per la linea seriale
 - ▶ La sequenza 1011 indica il comando per la media mobile

Realizzazione

► Le specifiche sono un po' vaghe

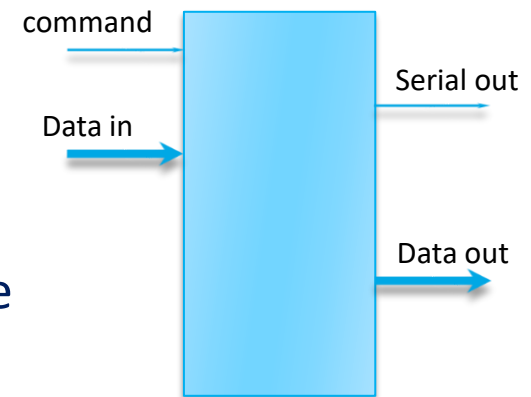
- Andiamo in modo incrementale per gradi
- L'oggetto avrà un ingresso seriale con cui riceve i comandi
- Avr  poi un ingresso parallelo con i dati
- Una uscita seriale in cui mandare il dato quando richiesto
- E magari una uscita parallela con la media mobile

► Occorre riconoscere i comandi

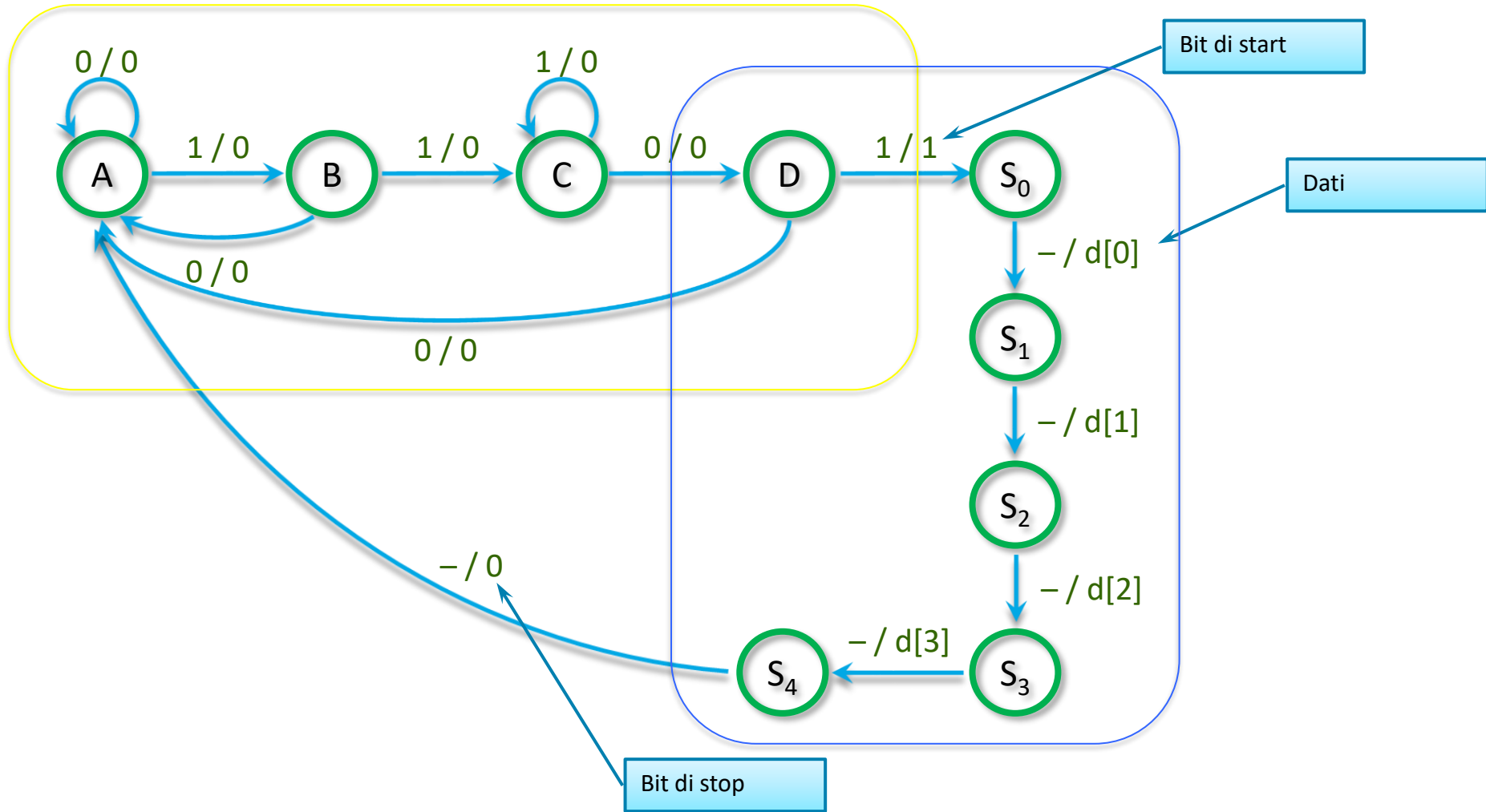
- Possiamo usare il nostro riconoscitore di sequenze
- Una volta riconosciuta una sequenza si esegue il comando

► Linea seriale

- Supponiamo di mandare i dati con un primo bit di start a 1, seguito dal resto dei bit, ed almeno un bit a 0 per lo stop



Gestione della linea seriale



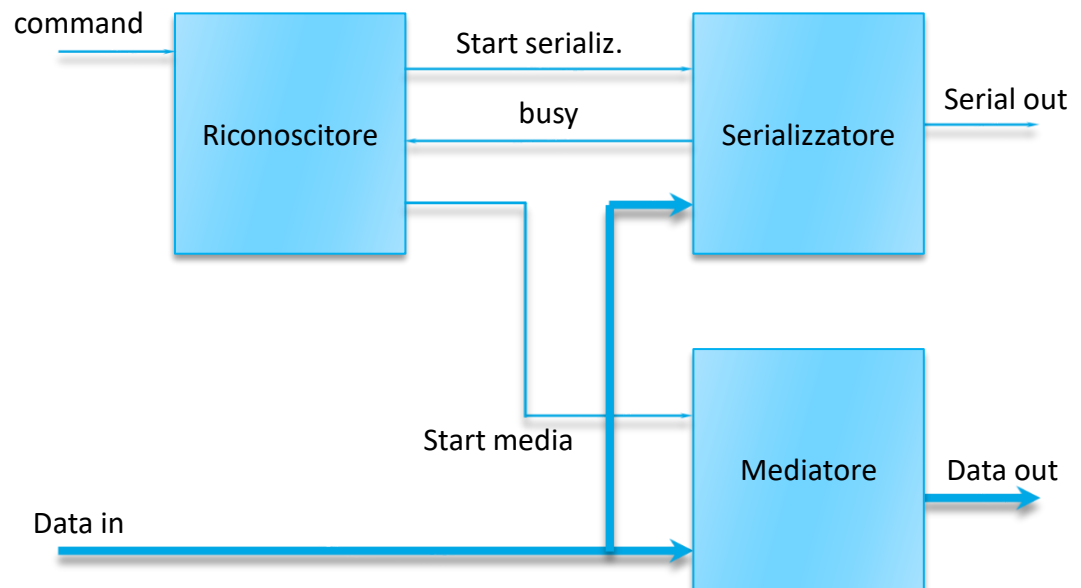
Comandi ravvicinati

- ▶ **E se arriva un altro comando mentre ne eseguiamo uno?**
 - ▶ La nostra macchina a stati non guarda la linea comandi mentre esegue la serializzazione
 - ▶ Nulla vieta però di farlo: occorre riprodurre il riconoscitore mentre si esegue la serializzazione
 - ▶ Ma quanto si complica la gestione?
 - ▶ Dobbiamo tenere conto di quello che si vede sulla linea comandi, e allo stesso tempo contare i bit che serializziamo
- ▶ **Meglio separare le due funzioni**
 - ▶ Un blocco riconosce, l'altro esegue
 - ▶ Occorre prevedere una sincronizzazione

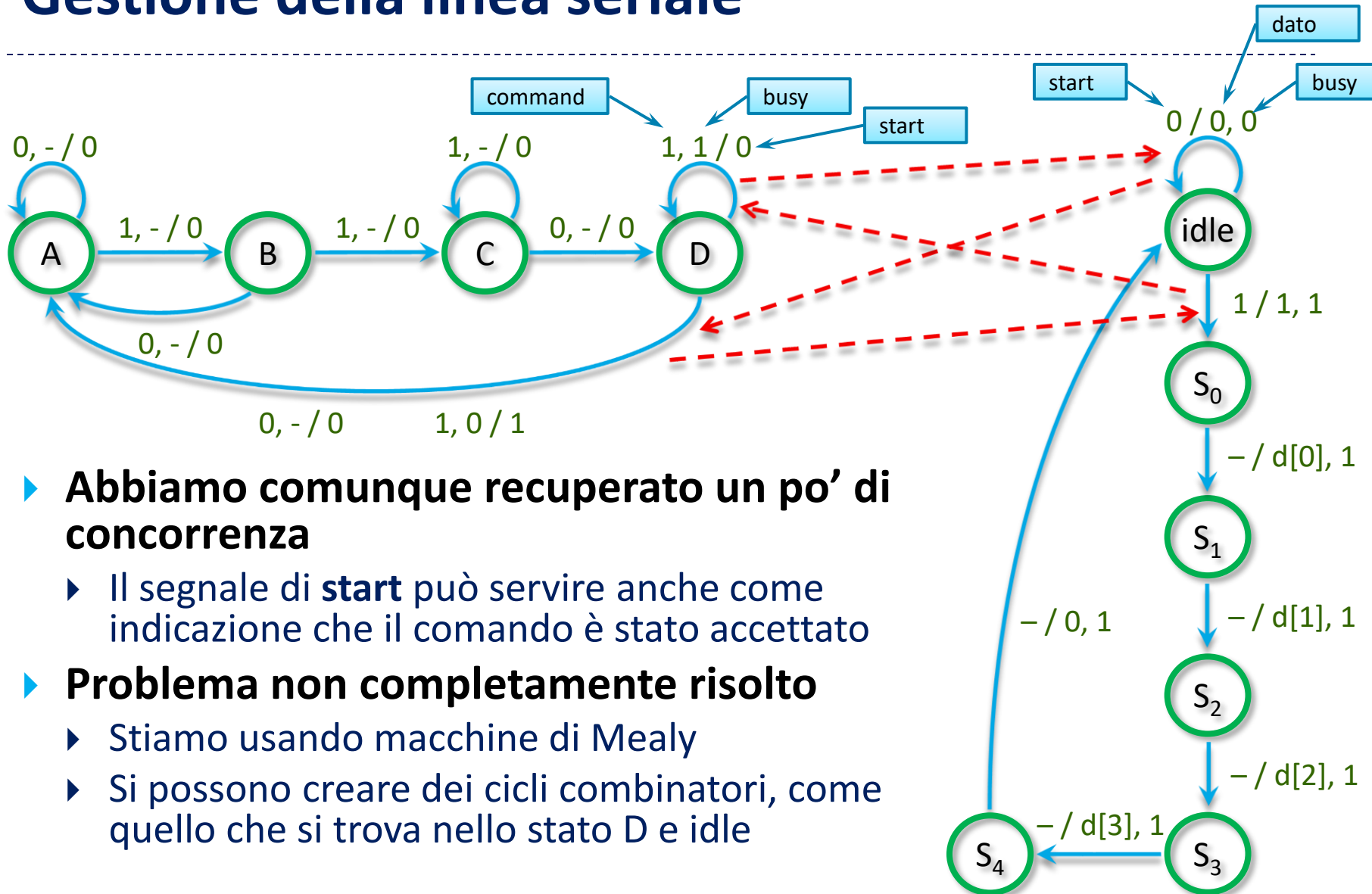
Partizionamento

► Scomponiamo il sistema in diversi blocchi

- Uno si occupa di riconoscere il comando
- Un altro di eseguire la serializzazione
- Un terzo di calcolare la media mobile
- Possono lavorare in parallelo

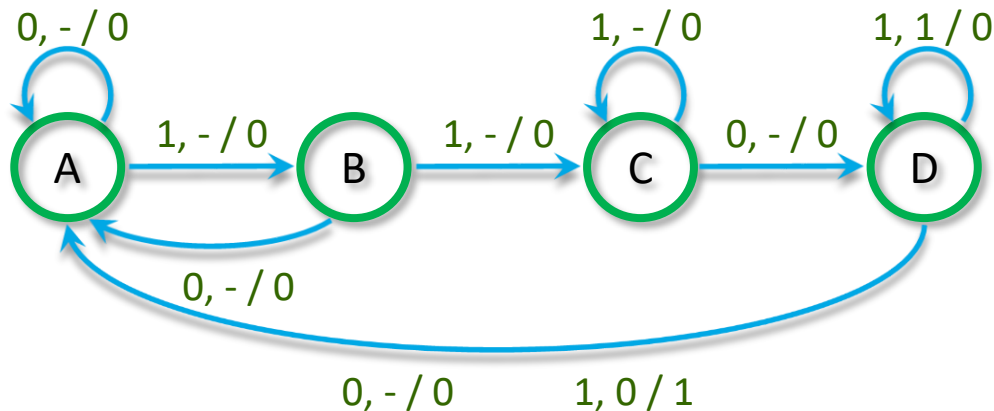


Gestione della linea seriale

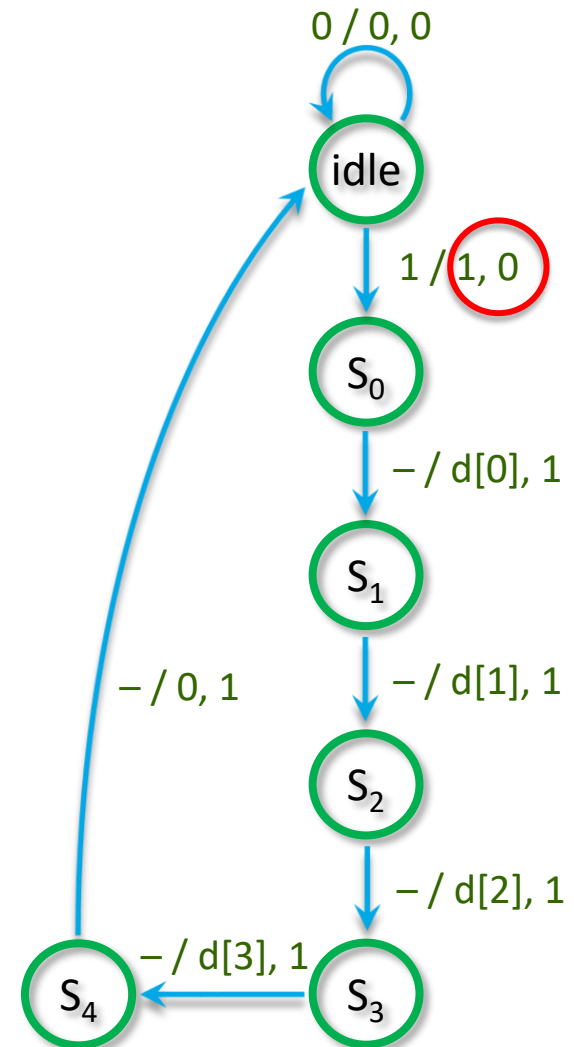


- ▶ **Abbiamo comunque recuperato un po' di concorrenza**
 - ▶ Il segnale di **start** può servire anche come indicazione che il comando è stato accettato
- ▶ **Problema non completamente risolto**
 - ▶ Stiamo usando macchine di Mealy
 - ▶ Si possono creare dei cicli combinatori, come quello che si trova nello stato D e idle

Soluzione



- ▶ Il segnale *busy* diventa dipendente solo dallo stato
 - ▶ In questo modo si frappa almeno un registro e non c'è più il problema
- ▶ Si può ancora pensare di memorizzare il dato
 - ▶ In questo modo si svincola il resto del sistema che può fornire altri dati in concorrenza
 - ▶ Ma comunque deve aspettare se si sta serializzando quello di prima



Realizzazione VHDL

- ▶ **Si fanno due macchine a stati separate**
 - ▶ Possono essere due entità distinte
 - ▶ Comunicano attraverso segnali di interfaccia
 - ▶ Oppure possono essere due processi distinti della medesima entità
 - ▶ Comunicano tramite segnali interni
 - ▶ Occorrono anche due segnali di stato separati
- ▶ **Stati totali**
 - ▶ Ci sono 4 e 6 stati nelle due macchine
 - ▶ Totale 24 stati nel sistema
- ▶ **Per casa, aggiungere il calcolo della media mobile**
 - ▶ Può essere utile suddividerlo in data path e controllo
 - ▶ Fossero comunque pure solo due stati, si ha in totale una cinquantina
 - ▶ Fondamentale in questo caso la divisione in componenti